Distributed Systems: Theory and Practice

Kishore Kothapalli
IIIT Hyderabad, India
kkishore@iiit.ac.in

Distributed Systems

- Efficient processing of large datasets has become essential to find answers to a range of questions involving scientific advances, drug discovery, planet-scale systems, and the like.
- Along with the availability of such datasets, there is a strong concomitant need to process such massive datasets in a scalable and efficient manner.

Distributed Systems

- One mechanism to achieve scale in such computations is to make use of distributed computation
- Here, the data resides across multiple machines.
- These machines can communicate via the provisioned communication links and perform the required computation through message passing and local computation.
- Facilitating such scalable solutions requires a multi-pronged approach.

Distributed Systems

- Facilitating such scalable solutions requires a multi-pronged approach.
- Firstly, there is a need for good programming models
  - Frameworks should allow programmers to express, write, debug, and execute distributed programs, all the while ensuring high levels of productivity, performance, and portability.
- Secondly, one needs strong algorithmic models to design and analyze distributed algorithms.
  - These algorithms should be closely aligned with practical considerations so that it is possible to translate such algorithms into efficient distributed programs.

Distributed Systems

- This two-pronged approach leads to a synergistic study of distributed systems and algorithms by blending theory and practice elements

Distributed Systems

- The Map-Reduce model stands out for its simplicity, expressibility, productivity, and portability.
- Unlike models such as MPI, the Map-Reduce model offers multiple advantages, including fault-tolerance.
- The Map-Reduce model also fits the cloud computing model naturally and represents related frameworks such as Hadoop and Apache Spark.

Distributed Systems

- Distributed computing is ripe with algorithmic models, too, including the LOCAL, CONGEST, Bulk Synchronous Parallel (BSP), k-machine, and Massively Parallel Computing (MPC).
- The Massively Parallel Computing (MPC) model offers a convenient way to design algorithms for various problems
- We will cover these two topics in the rest of this lecture(s).

Distributed Programming and Platforms

- Programming distributed systems poses multiple challenges
  - Divide a job into multiple tasks
  - Understand dependencies between tasks: Control, Data
  - Coordinate and synchronize execution of tasks
  - Send data between tasks
  - Stay clear of race conditions and deadlocks

- Need helpful framework that can help the programmer

Distributed Programming

- Recall that distributed systems are characterized as a collection of autonomous computers communicating over a network of links.
- Some typical features of such a system are:
  - No common physical clock
    - Clocks can drift and no notion of a common clock.
    - Systems can be asynchronous too.

Distributed Programming

- Recall that distributed systems are characterized as a collection of autonomous computers communicating over a network of links.
- Some typical features of such a system are:
  - No common physical clock
    - Clocks can drift and no notion of a common clock.
    - Systems can be asynchronous too.
  - No shared memory
    - Use messages for communication along with their semantics.

Distributed Programming

- Recall that distributed systems are characterized as a collection of autonomous computers communicating over a network of links.
- Some typical features of such a system are:
  - No common physical clock
    - Clocks can drift and no notion of a common clock.
    - Systems can be asynchronous too.
  - No shared memory
    - Use messages for communication along with their semantics.
  - Geographical separation
    - Can allow for wider scope of operations, e.g., SETI

Distributed Programming

- Recall that distributed systems are characterized as a collection of autonomous computers communicating over a network of links.
- Some typical features of such a system are:
  - No common physical clock
    - Clocks can drift and no notion of a common clock.
    - Systems can be asynchronous too.
  - No shared memory
    - Use messages for communication along with their semantics.
  - Geographical separation
    - Can allow for wider scope of operations, e.g., SETI
  - Autonomy
    - Processors are loosely coupled but cooperate with each other
  - Heterogeneity
    - All processors need not be alike.

Challenges of Distributed Programming

- Distributed programming is inherently challenging.
- Need to ensure that several geographically separate computers collaborate
  - efficiently,
  - reliably,
  - transparently, and
  - scalable manner
- One can also talk of inherent complexity and accidental complexity

Challenges of Distributed Programming

- One can also talk of inherent complexity and accidental complexity
- Inherent complexity due to
  - Latency: Computers can be far apart
  - Reliability: Recovering from failures of individual or collection of computers
  - Partitioning: Sometimes, failures can partition the system.
  - Ordering: Lack of global clock means message/event ordering is difficult
  - Security: Program as well as computational aspects

Challenges of Distributed Programming

- Distributed programming is inherently challenging.
- Need to ensure that several geographically separate computers collaborate
  - Efficiently, reliably, transparently, and scalable manner
- One can also talk of inherent complexity and accidental complexity
- Inherent complexity due to
  - Latency, Reliability, Partitioning, Ordering, Security
- Accidental complexity due to
  - Low-level APIs: Not enough abstraction
  - Poor debugging tools: Poor fault location
  - Algorithmic decomposition: Choice of algorithmic techniques
  - Continuous re-invention: Key components can change!

Enter Map-Reduce

- **MapReduce** is a distributed data-parallel programming model developed at Google
  - Introduced close to 20 years ago.
- MapReduce works best with a distributed file system, called **Google File System (GFS)**
- **Since then, there have been several variants**
  - **Apache introduced Hadoop** as an open source framework implementation that can execute the MapReduce programming model
  - Hadoop runs best with the **Hadoop Distributed File System (HDFS),** which is the open source implementation of the GFS design
  - Amazon's PaaS is yet another implementation of Map-Reduce

Map-Reduce

"A simple and powerful interface that enables automatic parallelization and distribution of large-scale computations, combined with an implementation of this interface that achieves high performance on large clusters of commodity PCs."

Dean and Ghermawat, "MapReduce: Simplified Data Processing on Large Clusters", OSDI, 2004

A High Level View

- Map Reduce provides
  - Clean abstraction for programmers
  - Automatic parallelization & distribution
  - Fault-tolerance
  - A batch data processing system
  - Status and monitoring tools

Map-Reduce

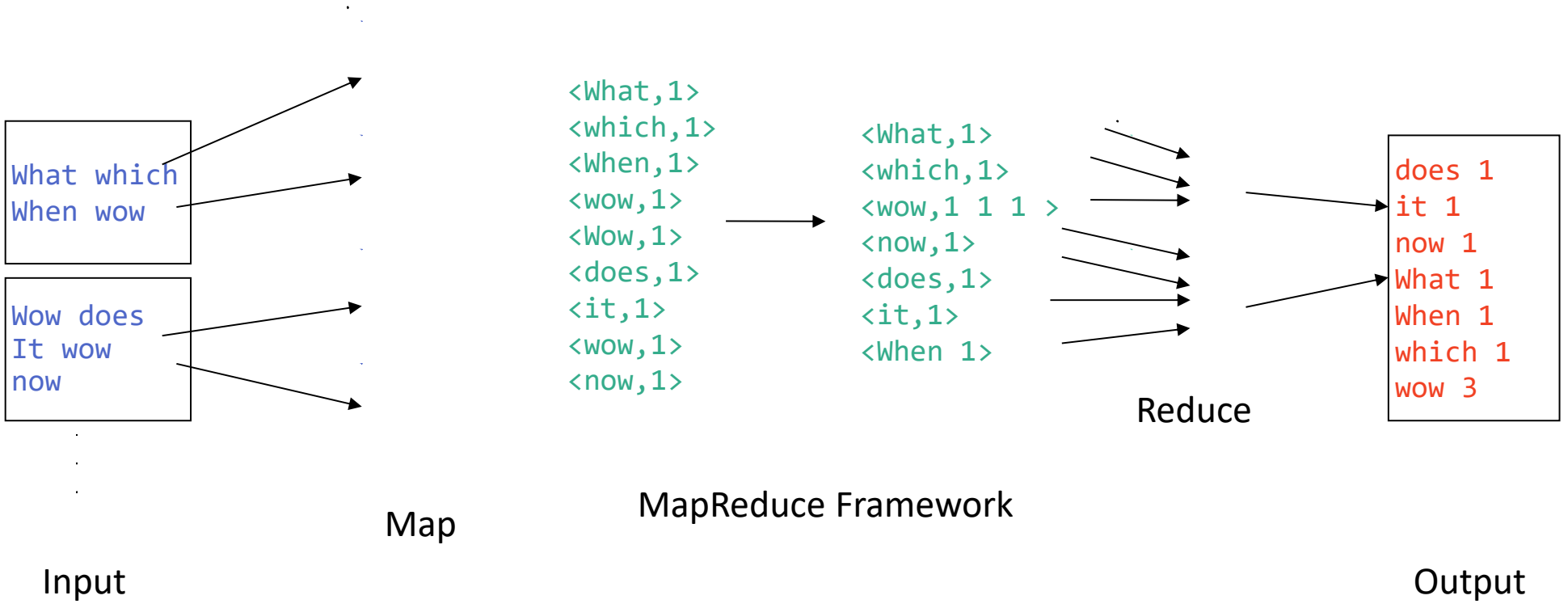
- MapReduce might sound like quite a restrictive programming model
- Mappers and reducers run with very limited coordination between one another.
- Typical problems that we can use Map-Reduce programming model are from image analysis, to graph-based problems, to machine learning algorithms.
- It can’t solve every problem, of course, but it is a general data-processing tool

MapReduce: Data-parallel Programming Model

- Process data using map & reduce functions
- **`map(`k_i`, `v_i`) → List<`k_m`, `v_m`>[]`**
  - ***map*** is called on every input item
  - Emits a series of intermediate key/value pairs
- All values with a given key are **grouped** together
- **`reduce(`k_m`, List<`v_m`>[]) → List<`k_r`, `v_r`>[]`**
  - ***reduce*** is called on every unique key & all its values
  - Emits a value that is added to the output

MapReduce: Word Count

Map(k1,v1) → list(k2,v2)
Reduce(k2, list(v2)) → list(k2,v2)



Map

- Input records from the data source
  - lines out of files, rows of a database, etc.
- Passed to map function as key-value pairs
  - Line number, line value
- map() produces *zero or more intermediate values*, each associated with an output key.

Map

- **Example: Upper-case Mapper**

```
map(k, v) { emit(k.toUpper(), v.toUpper()); }

("foo", "bar") → ("FOO", "BAR")
("Foo", "other") → ("FOO", "OTHER")
("key2", "data") → ("KEY2", "DATA")
```

- **Example: Filter Mapper**

```
map(k, v) { if (isPrime(v)) then emit(k, v); }

("foo", 7) → ("foo", 7)
("test", 10) → () //nothing emitted
```

Reduce

- All the intermediate values from map for a given output key are combined together into a list
- reduce() combines these intermediate values into one or more final values for that same output key … Usually one final value per key
- One output “file” per reducer

Mapper/Reducer Tasks *vs.* Map/Reduce Methods

- The user can specify the number of mapper and reducer tasks used by the application.
- Each **Mapper/Reducer task** can make multiple calls to **Map/Reduce method**, sequentially
- Mapper and Reducer tasks may run on different machines
- Implementation framework decides
  - Placement of Mapper and Reducer tasks on machines
  - Keys assigned to mapper and reducer tasks
  - But can be controlled by user…

Shuffle & Sort: *The Magic happens here!*

- **Shuffle** does a "group by" of keys from all mappers
  - Similar to SQL groupBy operation
- **Sort** of *local keys* to *Reducer task* performed
  - Keys arriving at each reducer are sorted
  - No sorting guarantee of keys across reducer tasks
- No ordering guarantees of values for a key
  - Implementation dependent
- Shuffle and Sort *implemented efficiently* by framework

Optimization: **Combiner**

- Logic runs on output of Map tasks, on the map machines
  - "Mini-Reduce," only on local Map output
- Output of Combiner sent to shuffle
  - Saves bandwidth before sending data to Reducers
- Same *input* and *output types* as Map's *output* type
  - `Map(k,v)` → `(k',v')`
  - `Combine(k',v'[])` → `(k',v')`
  - `Reduce(k',v'[])` → `(k'',v'')`

Optimization: **Partitioner**

- Decides assignment of intermediate keys grouped to specific Reducer tasks
  - Affects the load on each reducer task
- Sorting of local keys for Reducer task done after partitioning
- Default is hash partitioning
  - **`HashPartitioner(key, nParts)` → `part`**
  - Number of Reducer (**nParts**) tasks known in advance
  - Returns a partition number [0, nParts)
  - Default partitioner balances number of keys per Reducer … *assuming uniform key distribution*
  - May not balance the number of values processed by a Reducer

Combiner Advantage

- Mini-Shuffle lowers the overall cost for Shuffle
- E.g. *n* total items emitted from *m* mappers
- Reduces network transfer and Disk IO costs
  - In ideal case, `m` items vs. `n` items *written and read from disk*, *transferred over network* (`m<<n`)
- Shuffle, less of an impact
  - If more mapper tasks are present than reducers, higher parallelism for doing groupby and mapper-side partial sort.
  - Local Sort on reducer is based on number of unique keys, which does not change due to combiner.

MapReduce: Recap

- Programmers must specify:

map (k, v) → <k', v'>*

reduce (k', v'[]) → <k'', v''>*

All values with the same key are reduced together

- Optionally, also:

partition (k', number of partitions) → partition for k'

Often a simple hash of the key, e.g., hash(k') mod n

Divides up key space for parallel reduce operations

combine (k', v') → <k', v'>*

  - Mini-reducers that run in memory after the map phase
  - Used as an optimization to reduce network traffic

- The execution framework handles *everything else*…

What Does it Solve?

- Scalable data analytics
- Data not viewed or constrained by disk read/writes but find meaning in data via computation over sets of keys and values.
- Abstract out lots of programming details.
- Forgo removing data redundancy via normalization of tables.
  - The penalty of normalization is reading a record requires reading from multiple sources/tables/files.

“Everything Else”

- The execution framework handles everything else…
  - Scheduling: assigns workers to map and reduce tasks
  - “Data distribution”: moves processes to data
  - Synchronization: gathers, sorts, and shuffles intermediate data
  - Errors and faults: detects worker failures and restarts
- Limited control over data and execution flow
  - All algorithms must expressed in m, r, c, p
- You don’t know:
  - Where mappers and reducers run
  - When a mapper or reducer begins or finishes
  - Which input a particular mapper is processing
  - Which intermediate key a particular reducer is processing

Map-Reduce

- Failures are handled via the jobtracker and the tasktracker.
- The tasktracker notices failures of tasks including
  - Runtime exceptions
  - Sudden exit of the code executing the task
  - Tasks hanging – observed via timeouts
- If the jobtracker itself fails, Map-Reduce has no mechanisms to save and restore computations.

Models for Distributed Algorithms

- Having considered the distributed programming framework, let us now study algorithmic frameworks for distributed algorithm design.

Models for Distributed Algorithms

- We model the distributed system as a network or a graph G.
- The processors in the distributed system act as nodes of the graph G.
- The communication links between the processors act as the edges of the graph G.
  - Bidirectional links correspond to undirected graphs
  - Directional links correspond to directed graphs.
- Various topologies possible, but we leave it as *any* topology.

Nature of Execution

- Centralized Vs. distributed algorithms
  - Centralized: asymmetric roles; client-server configuration; processing and bandwidth bottleneck; point of failure
  - Distributed: more balanced roles of nodes, difficult to design perfectly distributed algorithms (e.g., snapshot algorithms, tree-based algorithms)
- Synchronous Vs. Asynchronous algorithms

Nature of Execution

- Synchronous Vs. Asynchronous algorithms
  - Synchronous:
    - upper bound on message delay
    - known bounded drift rate of clock wrt. real time
    - known upper bound for process to execute a logical step
  - Asynchronous: above criteria not satisfied
    - spectrum of models in which some combination of criteria satisfied
- Algorithm to solve a problem depends greatly on these assumptions.

System Model

- Distributed systems inherently asynchronous
- Algorithms must ideally be designed in the asynchronous model.
- However, most algorithm designers use the synchronous model.
- Take advantage of constructs called synchronizers that can convert synchronous algorithms to asynchronous versions.

Complexity Measures

- In a sequential algorithm too, several metrics of complexity are available.
  - Time
  - Space
  - Disk Accesses
  - ...
- Similarly, also in distributed systems, several notions of complexity exist.
- These notions may also change slightly based on the model of the system.

Complexity Measures

- In distributed systems, several notions of complexity exist.
- These notions may change slightly based on the model of the system.
- Typical measures of complexity include:
  - Space complexity per node
  - System-wide space complexity
  - Time complexity per node
  - System-wide time complexity. Do nodes execute fully concurrently?
  - Message complexity
    - Number of messages (affects space complexity of message overhead)
    - Size of messages (affects space complexity of message overhead + time component via increased transmission time)
  - Message time complexity: depends on number of messages, size of messages, concurrency in sending and receiving messages

Algorithmic Primitives

- Some algorithmic steps are known to be powerful primitives, a. la. subroutines, that can be useful in distributed algorithm design.
- Two such are: Broadcast and Convergecast.
- Broadcast: distribute information
  - Step 1: Root sends info to be broadcast to all its children. Terminate.
  - Step 2: When a (nonroot) node receives info from its parent, it copies it and forwards it to its children. Terminate.

Algorithmic Primitives

- Some algorithmic steps are known to be powerful primitives that can be used as subroutines.
- Broadcast and Convergecast are two such routines.
- Convergecast: collect info at root, to compute a global function
  - Step1. Leaf node sends information to its parent and terminates.
  - Step2. At a non-leaf node that is not the root: On receiving information from all its children, send (fused) information to the parent and terminate.
  - Step3. At root: On receiving information from all its children, evaluate the necessary global function and terminate.

Algorithmic Primitives

- Uses: compute min, max, leader election, compute global state functions
- Time complexity: $O(h)$; Message complexity: $n - 1$ messages
- In the above, h is the height of the tree used.

Models for Distributed Algorithms

- LOCAL
- CONGEST
- CONGESTED-CLIQUE
- Massively Parallel Computing Model

LOCAL Model

- Each round consists of local computation and message send/receive events.
- Local computation, polynomial or more, is completely ignored in understanding the complexity.
- The volume/size of messages are not taken into account.
- Any node can send a message of size polynomial in the input size in one round to each neighbor.
- The parameter of efficiency of an algorithm is the number of rounds required to complete.

LOCAL Model

- For most graph problems, a trivial upper bound therefore would be the diameter of the graph.
- Each node sends its list of neighbors to a special designated nodes, say node number 1.
- Node number 1 receives the neighbor lists from each node. It can then assemble the graph.
- Node number 1 can then compute the required answer and if needed, send the answer to all other nodes.
- However, it is possible to do better in many cases.

LOCAL Model

- For MIS for instance, one can show an upper bound of O(sqrt{log n} log Δ) rounds in the randomized rounds.
  - Algorithms are difficult to analyze.
  - A trick used in such fast algorithms is to run a usual slow algorithm for a few rounds.
  - At the end of these few rounds, carefully analyze the progress made by the algorithm.
  - Obtain bounds on the properties of the left over graph.
  - Use the fact that message sizes can be large to gather the graph at a single node, solve locally, and distributed the answer.
- Lower bounds are much more tricky to prove.
  - Lower bounds for a few problems are known.

CONGEST Model

- Here too, the network representing the distributed system is abstracted as a graph.
- The nodes of the network correspond to the nodes in the graph, and the communication links are the edges of the graph.
- In a departure from the LOCAL model, here it is assumed that the bandwidth of the links is limited.
- So, messages are limited in size and bigger messages require more rounds.
- The bandwidth of each link is usually limited to O(log n) where n is the number of nodes in G.

CONGEST Model

- In a departure from the LOCAL model, here it is assumed that the bandwidth of the links is limited.
- So, messages are limited in size and bigger messages require more rounds.
- The bandwidth of each link is usually limited to O(log n) where n is the number of nodes in G.
- O(log n) is the size required to send the id of a node of G with n nodes. Hence, the limit is not artificial.

CONGEST Model

- Non-trivial lower bounds are not known for many problems.
- Algorithm design also more difficult than the LOCAL model.

- For example, Luby‘s MIS algorithm can be made to run in the CONGEST model.

CONGESTED-CLIQUE

- This model is significantly different from the LOCAL and the CONGEST models.
- The model uses a clique graph of n nodes with communication links connecting each pair of nodes.
- This model is one of the so called All-to-All Communication models.
- The input graph is a spanning subgraph of this graph.
  - In other words, the input graph too has n nodes but possibly fewer links.
- However, all communication links can be used to solve the problem of interest.

CONGESTED-CLIQUE

- The input graph is a spanning subgraph of this graph.
- However, all communication links can be used to solve the problem of interest.
- Each link however has a limited bandwidth of O(log n) bits.
- The extra communication links can be used to quickly send information but the size of the message in each round is limited.

Some Open Issues

- The CONGESTED-CLIQUE model is a bit too relaxed and artificial.
- Usually, expanding the communication network as a clique is difficult. Not scalable in practice.
- However, one can create what are called as expanders easily
- An expander graph has the property that any two n/2 sized subsets of the graph have a good connectivity between them
  - Such graphs with low degree, as low as O(1), also exist!

A Model for Cloud Computing

- It is important to connect distributed computing models to practical models like the cloud.
- One step in that direction is the Massively Parallel Computing model, MPC model for short.
- The MPC model has a number of pairwise interconnected machines.
  - Similar to the All-to-All communication model
- However, we assume that no single machine can store the entire input by itself.
  - Space is not enough!
- We still measure the number of rounds of communication needed to solve a problem.

The MPC Model

- Let the input have a size of N.
- Let each machine have a space of S. We assume that N >> S.
- Each machine can in each round send and receive up to S units of data via its links.
- The input is distributed across the machines in equal proportion with each of the O(N/S) machines holding O(S) data.
- When the input is a graph, notice that N = O(m+n).
- There are three variants studied in this context based on the relation between n and S.

A First MPC Algorithm: Sorting

- Consider a large dataset that is to be sorted.
- We will set N to be the number of items to be sorted.
- Note that N >> S, and hence the data is spread across the N/S machines with each machine hosting S items.
- We will design an algorithm akin to quicksort for this problem.

MPC Sorting

- We will let each machine mark each element it hosts as a pivot with a probability p, indepdently.
- The number of pivots in each machine on expectation is p.S.
- We gather all the pivots to a single machine, say M.
- At M, we sort the pivots and pick a subset of pivots such that there is at least one pivot for every n/p ranks of the input.
- Machine M sends the chosen pivots to all other machines.

MPC Sorting

- Each machine now partitions the input items it has according to the chosen pivots.
- Each machine M arranges the items it has into buckets based on the pivots.
- Machines exchange items so that each machine has items that lie between two pivots.
- Each machine can then sort the items locally.

Analysis

- We have to see how to choose p so that
  - There are not too many pivots
  - There is a pivot for every set of elements.

Analysis

- Lemma: With high probability, some machine marks at least one element in every segment of N^{δ} ranks of the input A.
- Proof. The probability that no machine marks any of the elements in a given segment of N^{δ} ranks of the input array is $(1 - 1/p)^{mnp} <= \exp\{-mn\} = e^{-N}$.
  - Here, m is the number of machines and n is the number of items in a machine.

Analysis

- We analyze the number of communication rounds as follows.
- Each machine selects O(n.p) pivots with high probability.
- The total number of words that a machine receives is in $O(m \cdot n \cdot p) = O(N \cdot p)$.
- We set p such that $N \cdot p = S$, or $p = S/N$.
- At this value of p, the communication volume at each machine is in O(S).
- Further, the last communication step of rearranging elements also has a communication volume of O(S).

Lessons Learnt

- Local computation is a way to save on communication.

The MPC Model

- The MPC model is more meaningful when applied to graph inputs.
- When the input is a graph, notice that $N = O(m+n)$.
- There are three variants studied in this context based on the relation between n and S.
  - The super-linear model: $S = O(N^{1+\varepsilon})$, for some $\varepsilon > 0$.
  - The near-linear model: $S = O(N)$
  - The sub-linear model: $S = O(N^{1-\varepsilon})$ for some $\varepsilon > 0$.

- Let us study how we can design algorithms for the MST problems in the above three regimes.

Minimum Spanning Tree

- Given a weighted connected graph on n nodes, find a spanning tree of minimum total weight.
  - The tree that minimizes the sum of the weights of the edges in the spanning tree.
- Traditional algorithms include
  - Kruskal's algorithm
  - Prim's algorithm
- These do not work well in the distributed setting.

The MPC Model

- The super-linear regime: Here, $S = \Omega(n^{1+\varepsilon})$.
- We still assume that $m >> S$.
- So, each machine has enough space to store information about all vertices and more than that.
- How is this additional space helpful?
- Partition the edges of the graph into $E_1, E_2, \ldots, E_r$ such that $|E_i| = n^{1+\varepsilon}$.
- Compute $T_i = MST(E_i)$.
- Recursively compute and return the MST of $U_i T_i$.

The MPC Model

- How many recursive calls are needed?
- Assume $m = n^{1+c}$.
- So, initially, there are $m/n^{1+\varepsilon} = n^{c-\varepsilon}$ sets of edges.
- After computing T_i with at most n edges each, the number of edges in $U_i\ T_i$ is at most $n x n^{c-\varepsilon} = n^{1+c-\varepsilon}$.
- So, each recursive call reduces the number of edges from n^{1+c} to $n^{1+c-\varepsilon}$.
- In c/ε such recursive calls, the number of edges falls to a quantity where a single machine can store all the edges.
- Algorithms that run in $O(1/\varepsilon)$ rounds are known for several other graph problems too in this model.

The Linear MPC Model

- Having $S = n^{1+\varepsilon}$ makes it very easy to design algorithms that are super-fast.
- Let us reduce S to $\Theta(n)$. We call this the linear MPC model.
- In the linear model, each machine has enough space to store a very limited information about each node in the graph.
- Let us continue the MST example and see how to design algorithms in this model.

The Linear MPC Model

- The techniques needed here are quite involved.
- Consider a sorted order of edges by weight. Let the ordering be $e_1, e_2, \ldots, e_m$.
- The following observation holds:

  Edge e_i is in the MST if and only if its endpoints are not in the same connected components in the graph with edge set $\{e_1, \ldots, e_{i-1}\}$.
- We can use this observation directly to convert the MST problem to the connected components problem.
  - Check the observation for each of e_1 to e_m.
- However, this algorithm is quite wasteful.

The Linear MPC Model

- One improvement is as follows.
- Group the m edges into m/n groups of n edges each.
- Call $E_i = \{e_{(i-1)n+1}, \ldots, e_{in}\}$ for i = 1 to m/n.
- Let $H_i = \cup_{j=1}^{i} E_j$. H_i is the union of the first i E_is for i=1 to m/n.
- Let F_i be the MST (Forest) corresponding to H_i.
- Notice that $|F_i| = O(n)$.
- So, given F_i and E_{i+1}, both of size O(n) edges, one machine can check which edges in E_{i+1} will be in the MST.
- We could compute each F_i in parallel!

The Linear MPC Model

- Call $E_i = \{e_{(i-1)n+1}, \ldots, e_{in}\}$ for i = 1 to m/n.
- Let $H_i = U_{j=1}^{i} E_j$. H_i is the union of the first i E_is for i=1 to m/n.
- Let F_i be the MST (Forest) corresponding to H_i.
- Notice that $|F_i| = O(n)$.
- So, given F_i and E_{i+1}, both of size O(n) edges, one machine can check which edges in E_{i+1} will be in the MST.
- We could compute each F_i in parallel!
- But, $|H_i|$ is too big to fit in any single machine!!

The Linear MPC Model

- We just wish that computing the F_is takes as few rounds as possible.
- Here is a way to do that.
- We use the technique of graph sketching.
- We then explain how to implement the technique in the linear MPC model to compute connected components in O(1) rounds.

The Linear MPC Model

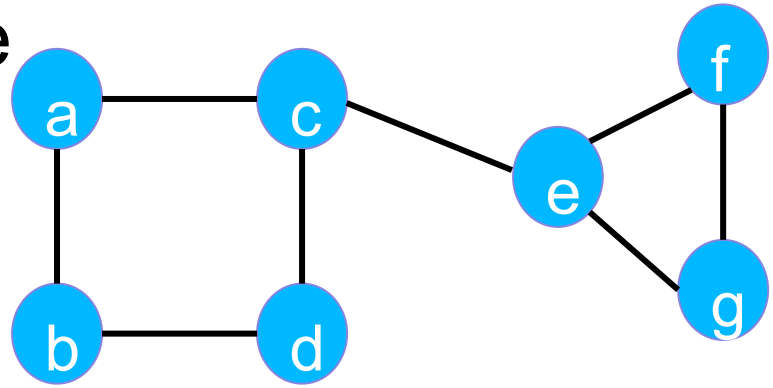
- Let A be a subset of V. Let C be the cut between A and V \ A.
- Assume for a moment that the cut has only edge crossing it.
- We wish to find the cut edge across A and V \ A using only O(log n) bits of memory.
- We use a bit trick that works as follows.

The Linear MPC Model

- Identify each edge by the concatenation the identifiers of its endpoints. For e = uv, id(e) = u o v if id(u) < id(v).
- Locally, each vertex u computes
  $XOR_u = \bigoplus_{e \in E, e \text{ has u as an end point}} id(e)$.
- Now, consider the XOR of all the vertices in A, $XOR_A = \bigoplus_{u \in A} XOR_u$.
- In XOR_A, an edge e = uv with both end points A appears in both XOR_u and XOR_v.
- On the other hand, the edge e = uv with u in A and v in V \ A, appears only once (in XOR_u).
- So, the result of XOR_A = id(e) thereby allowing us to identify the cut edge.

The Linear MPC Model

- Example



Edge	End points	ID
e1	a, c	000010
e2	a, b	000001
e3	b, d	001011
e4	c,d	010011
e5	e, f	100101
e6	f, g	101110
e7	e, g	100110
e8	c, e	010100

Node	Edges	XOR_u
a	e1, e2	000011
b	e2, e3	001010
c	e1, e4, e8	000101
d	e3, e4	011000
e	e5, e7, e8	100111
f	e5, e6	001011
g	e6, e7	111000

XOR_A = XOR(000011, 001010, 000101, 011000) = 010100 = ID(e8)

The Linear MPC Model

- We need to extend the above idea for the case where the cut has multiple cross edges.
- If the cut has more than one edge, then all such edges appear only once in XOR_A.
- The resulting XOR_A may actually end up being the id of some edge that is not a cut edge!
- So, we need a couple more techniques.

The Linear MPC Model

- Our first technique is to replace the id of vertices with O(log n) bit random strings of {0, 1}.
- With this, Pr(There exists an edge e, XOR_A = id(e)) <= Σ_e Pr(XOR_A = id(e)) <= ${}^nC_2$ x $(½)^{12\log n}$ = $1/n^{10}$.
- This low probability helps us tide over the trouble of using actual vertex identifiers.

The Linear MPC Model

- For our second technique, we do the following.
- We do not know the number of edges crossing the cut.
- But, the number lies between 0 to n.
- So, we will try all possible estimates for the above number. All in parallel!
- The estimate is good if the actual number of edges crossing the cut is within ½ to 2 of the estimated number.
- In essence, we try estimates such as {1, 2, 4,…, n}.

The Linear MPC Model

- Let k be the actual number of cut edges.
- Run each estimate k' in [1, 2, 4, …, n] in parallel.
- With a given k', we mark edges with a probability of 1/k'.
- Consider the k' such that k'/2 <= k <= k'
- With a marking probability of 1/k', the expected number of cut edges marked is 1.
- We can reuse the case of identifying one cut edge if we get to mark just one of the k cut edges and no other edges are marked.
- Let us calculate the probability for the above event.

The Linear MPC Model

- Let S be the set of cut edges marked.
- $\Pr(|S| = 1) = k \,.\, (1/k') \, (1-1/k')^{(k-1)}$

  $>= (k'/2) \,.\, (1/k') \, (1-1/k')^{(k-1)}$.

  $>= (1/2) \,.\, (1/4)$ as $1 - x >= 4^{-x}$.

  $>= 1/8$.

The Linear MPC Model

- A few more nitty-gritty details are needed beyond the two techniques.
- In particular, we will need multiple independent runs with each k'.
- With a success probability of at least 1/8, Chernoff bounds tell us that O(log n) runs will help us.
- The overall technique is called graph sketching.

The Linear MPC Model

- Since the memory needed per sketch per node is O(log n), the linear MPC model can support the technique.

- Putting together everything, we get:
- The MST of a graph G can be obtained in O(1) rounds in the linear memory MPC model.

The Sub-linear MPC Model

- In the sub-linear MPC model, the only best known solution for the MST problem so far is to use the Boruvka algorithm.
- We present a brief overview of the GHS algorithm.
  - We will use Broadcast and Convergecast in designing this distributed algorithm.
  - Later, we will talk about simulating it in the MPC model with sublinear memory.

Minimum Spanning Tree

- Let T be the MST of a graph G = (V, E, W)
- Some common notions in most distributed MST algorithms are:
  - An MST fragment F of T is a connected subgraph of T.
  - An outgoing edge of an MST fragment F is an edge e in E such that one end point of the e is in F and the other is not.
  - The minimum-weight outgoing edge (MOE) of a fragment F is the edge with minimum weight among all outgoing edges of F.

Minimum Spanning Tree

- Check for yourself: The MOE of a fragment F = (V_F, E_F) is an edge of the MST T.
- The GHS algorithm operates in phases.
- In the first phase, the algorithm starts with each individual node as a fragment by itself.
- At the end, there is only one one fragment which is the MST.
- All fragments find their MOE's simultaneously in parallel.

Minimum Spanning Tree

- Invariant across Phases: Each MST fragment has a leader and all nodes know their respective parents and children.
- The root of the tree will be the leader.
- Each fragment is identified by the identifier of its root called the fragment ID.
- Each node in the fragment knows its fragment ID

Minimum Spanning Tree

- Here is how one Phase of the algorithm operates.
- Two operations in each phase
  - Find MOE of all fragments, and
  - Merging fragments via their MOEs.

Finding MOE

- The root of the fragment broadcasts a message <Find MOE> to all nodes in the fragment using the edges in the fragment.
- When a node receives a <Find MOE> message, it finds its minimum outgoing incident edge, i.e. the minimum weight outgoing edge among all the incident edges.
  - This can be done by checking the neighbors of the node.
- Then, each node sends its minimum outgoing incident edge to the root by using the ConvergeCast routine.
- The root then finds the MOE as the minimum among all the edges convergecast.

Merging Fragments

- In this phase, fragments are merged via their MOEs.
- Once the leader finds the MOE, it broadcasts a <Merge MOE> message to all its fragment nodes
- When a node receives the Merge message, it knows whether it is has the MOE edge incident on it.
- If so, it sends a “Request to combine" message to its neighbor which is the other end point of the MOE edge.

Minimum Spanning Tree

- The node with the higher identifier becomes the root of the combined fragment.
- The (combined) root broadcasts a <NEW-FRAGMENT> message through the fragment edges and the MOE edges chosen by all the fragments.
- Each node updates its parent, children, and fragment identifier.
- Note that since each fragment has only one outgoing edge, there can at most one pair of neighboring nodes

Analysis

- The total number of phases is O(log n).
- This is because, in each phase, the total number of fragments is reduced by at least half.
- Each phase takes O(n) time.
  - All messages travel along the MST.
  - Diameter of the MST is at most O(n)
- Each phase takes O(n) messages plus the messages needed to find MOE.
- O(m + n log n) messages because in each phase a node checks its neighbor in increasing order of weight starting from the last checked node

Final Analysis

- The GHS algorithm described above correctly a distributed MST in O(n log n) rounds and uses O(m + n log n) messages.

MPC Implementation

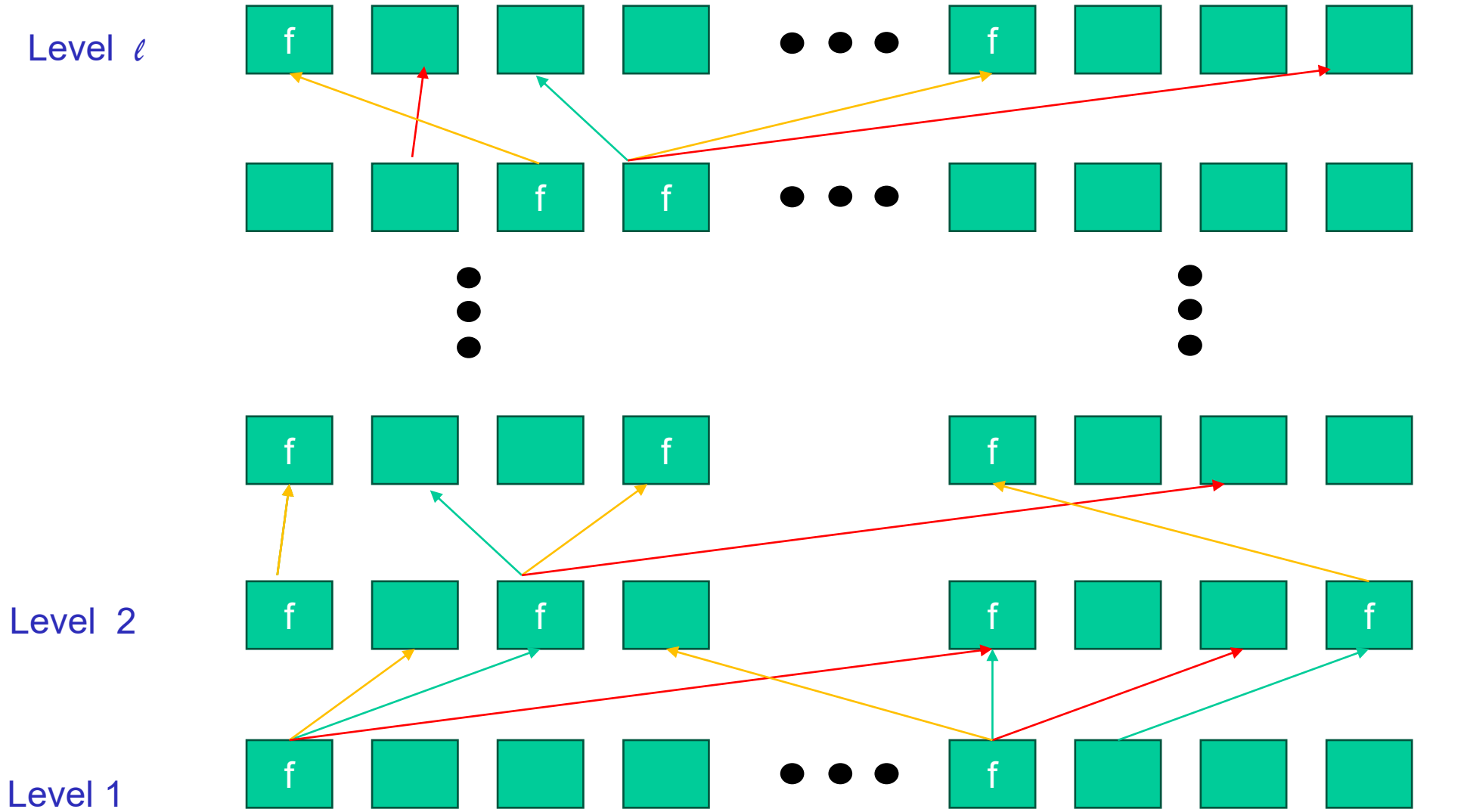
- The real challenge is to see how each phase of the algorithm can be implemented in O(1) rounds of the low-memory MPC model.
- All-to-all communication does not help since the volume of communication can be more than $O(n^{\delta})$.
- Another issue could be a long chain of fragments that merge into one another.
- We solve these two problems using randomization.

MPC Implementation

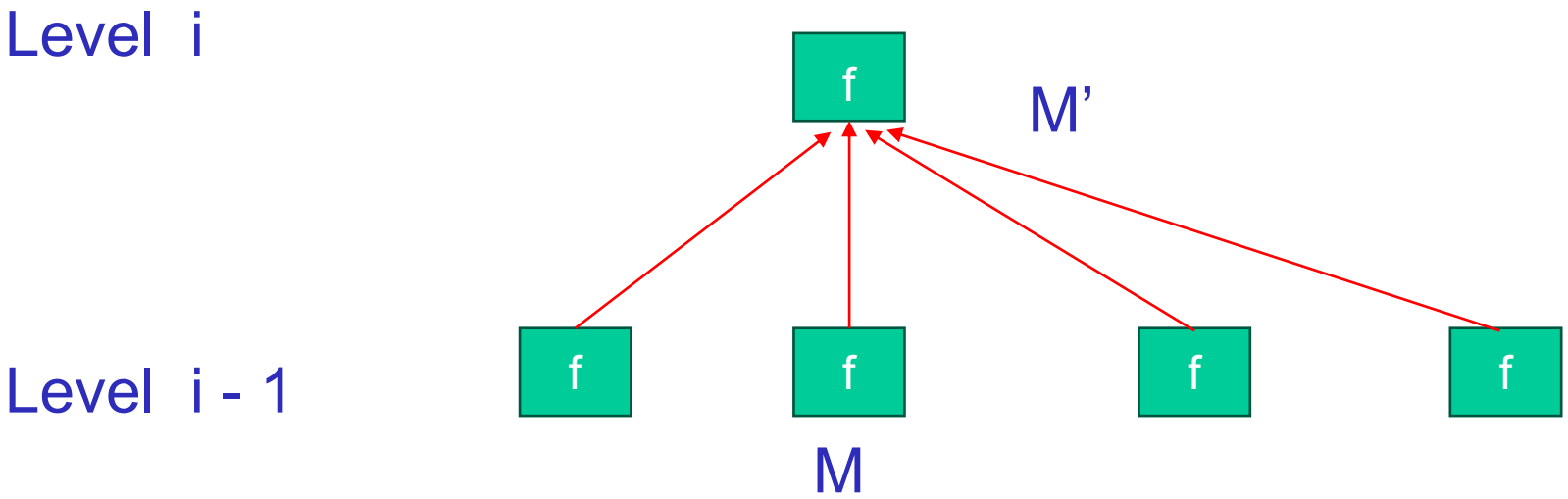
- Arrange the set of machines such that at level 1, we have all the m/S machines, each storing S edges.
- We say that a machine M works for an MSF fragment F if M stores an edge in F.
- At level 1, machine M works for a fragment F if M has an edge uv that belongs to fragment F.
- For a level i > 1, we say that a machine M on a level i works for a fragment F if M participates in the computation for the fragment F.

MPC Implementation

- Across two consecutive levels i and i+ 1, with $i \geq 1$, define a parent-child relationship across machines as follows.
- Let ℓ be the largest level number
- For each fragment F and each level i for $1 \leq i \leq \ell$, each machine M on level i that works for fragment F has a parent machine M′ with respect to fragment ID F on level i + 1 if M′ also works for fragment F.
- On level $i \geq 2$, a machine M is responsible for fragment F if a machine M exists on level $i - 1$ such that M is the parent with respect to fragment F of level $i - 1$.


Level ℓ
Level 2
Level 1
f

MPC Implementation

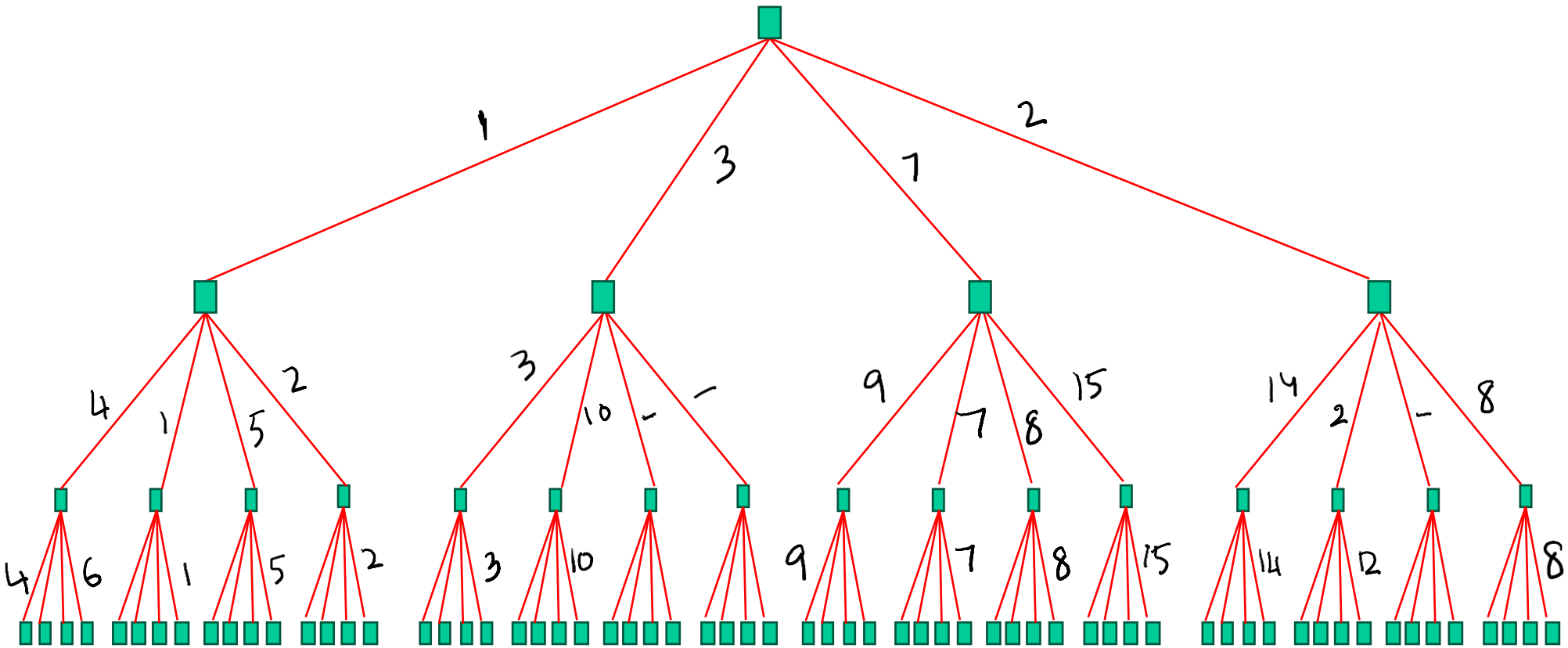


- The in-degree of a machine M′ on level i > 1 is the number of pairs (M, F) such that M′ is the parent of machine M with respect to fragment F on level i − 1.

MPC Implementation

- For each fragment F, each machine M on level 1 sends the minimum weight edge with exactly one endpoint in F that it stores to its parent.
- Continuing this level-by-level traversal, each machine on a level $i \geq 2$ waits to receive information from its children i level $i-1$.
- They can then send the required information to their parent with respect to each fragment F.
- This is akin to the convergecast operation.

MPC Implementation



MPC Implementation

- Further, let uv be the minimum weight each that the machine on the top level receives for a fragment F, with u in F and v in another fragment F′ .
- Imagine that all the fragments are colored blue and red with equal probability.

MPC Implementation

- At the last level, ℓ, for an edge uv to be added, the machine checks whether u is in a red-colored fragment F and v is in a blue-colored fragment F′.
- Only then, the machine in level ℓ adds edge uv to the MST and initiates the change of representative from fragment F to that of fragment F′.
- This machine then broadcasts this change in representative to its children with respect to fragment F.
- This is akin to using the broadcast operation in the GHS algorithm.

Analysis

- Claim 1: The number of levels, ℓ, is= O(1).
- Proof: The structure has a fan out of m/4, the number of levels is in $O(\log_{S/4} m) = \log m/\log(S/4)$.
- Since $S \in n^{\delta}$, the ratio is in $O(4/\delta)$ provided $S > 16$.

- Claim 1 implies that the upward and downward traversals along the structure require O(1) communication rounds.

Analysis

- Claim: The in-degree of any machine at any given level is O(S).
- Proof: To this end, consider a machine M in a level $i \geq 1$, and a fragment F.
- Note that $m/(S/4)^i$ machines are in level i.
- So, the probability that machine M has to work for fragment F is $m/(S/4)^i$ / $m/(S/4)$) which is $(4/S)^{i-1}$.
- Given that M is working for fragment F, we bound the expected number of children for M as the ratio of the number of machines in level $i - 1$ working for fragment F over the number of machines in level i working for fragment F.
- This ratio is S/4, which is the fanout of the structure.

Analysis

- Continuing the proof of Claim 2, the unconditional expected number of children of M with respect to fragment F is thus $(4/S)^{i-1} \times (4/S) = (4/S)^{i}$.

- We now bound the expected in-degree of machine M as the product of the number of fragments with the expected number of children with respect to a given fragment.
- Thus, the expected indegree of machine M is at most $n \times (4/S)^{i-2} = S/2$, as $S = n^{\delta}$.
- Claim 2 helps also bound the communication volume at any machine to be within S.

Analysis

- Using Claims 1 and 2, we infer that each phase of the algorithm can run in O(1) rounds.

Summary of the MPC model and MST

- Super-linear memory: $O(1/\varepsilon)$ rounds, easy technique.
- Linear memory: $O(1)$ rounds, but very complicated and lots of techniques.
- Sub-linear memory: $O(\log n)$ rounds, with a complicated simulation and analysis.

MPC Model – Other Graph Algorithms

- Wide body of literature wrt MPC algorithms for various graph problems including MIS.
- Many of these results use randomization.
- Recent successes include derandomization as well.
- Variants of the MPC model also under study.
- A good thesis topic with lots of interesting problems yet to be solved.

The Practice ←→ Theory Bridge

- How do the algorithm model and the programming platform serve to bridge the theory and practice of distributed systems?
- The MapReduce system does not explicitly limit the space per machine.
- The framework does not have a mechanism to count the number of rounds of communication needed.
- The framework also does not place any theoretical limit on the space that each machine has.
- The framework typically is used to study the run time the program takes on a given input with a specified number of mapper and reducer machines

The Practice ←→ Theory Bridge

- The MPC model, in contrast, enforces a discipline on algorithm designers to design algorithms that respect the space limit on any individual machine.
- By doing so, we expect that the algorithm and the corresponding implementation in a Map-Reduce framework continue to work on varying input sizes

MPC ←→ MapReduce

- Lattanzi et al. suggest that one can analyze Map-Reduce programs for the number of rounds by considering one Map function call followed by one Reduce function call as one round.
- Lattanzi et al. argue that implementing every graph algorithm designed in the superlinear memory MPC model as O(1) round algorithms in the Map-Reduce model is possible.
  - Here, O(1) rounds refer to the number of map and reduce rounds that the program in the Map-Reduce framework needs.
- However, their work is limited to graph algorithms and the superlinear regime.

MPC ←→ MapReduce

- The Map-Reduce framework requires programmers to express every computation as a sequence of map and reduce functions, each of which works on (key, value) pairs.
- Expressing every computation through map and reduce functions is not possible.
- On the other hand, we express MPC algorithms using general-purpose statements, and it is possible to write every such MPC algorithm as a suitable program in a distributed message-passing system

MPC ←→ MapReduce

- To summarize, it is possible to think of MPC algorithms and Map-Reduce programs as the two sides of the theory and practice of distributed systems, a few gaps still exist.
- Ideally, practical frameworks should include provisions for limits on space and network bandwidth.
- Similarly, algorithmic models such as the MPC should look at optimizing not only the round complexity but also the overall work done by the algorithm.
- Another approach to pursue would be to arrive at a combination of parameters that help bridge the theory and practice of distributed systems.